

Vorbereitung Live-Demo (20.05.2026)

Was ich in der Demo zeigen will

Ich führe die Demo in vier Blöcken durch: 1. kurzer Einstieg mit Spezifikation und Use Cases, 2. Live-Bedienung der Anwendung, 3. Tests, Coverage und Javadoc, 4. besondere Features (UI/UX, Theme-Sync, Easter Egg im PDF).

Kurzcheck vor dem Termin

```
cd /Users/simonriedinger/dev/klausurCraft/demo
./scripts/live-demo-check.sh
```

Optional mit neuem Coverage-Lauf:

```
./scripts/live-demo-check.sh --coverage
```

Dokumente in sinnvoller Reihenfolge

1. abgaben/SpezifikationKlausurCraft.pdf
2. abgaben/UseCases.png
3. abgaben/GlassBox_Coverage_Report.pdf
4. abgaben/Metriken_Report.pdf
5. abgaben/Zufallsbasierter_Test_Report.pdf
6. abgaben/Refactoring_Report.pdf
7. abgaben/Tickets.pdf
8. abgaben/UI_UX_ELEGANCE_Report.pdf
9. abgaben/javadoc-report-2026-04-22/index.html

Konkreter Ablauf (mit Zeitgefühl)

1) Einstieg (2–3 Minuten)

- Projektziel in 2 Sätzen erklären.
- Use Cases zeigen.
- Kurz sagen, welche Teile fachliche Logik sind und welche Teile UI sind.

2) Live-Teil in der App (8–10 Minuten)

- Starten: `mvn -q javafx:run`
- Vorführen:
 - neue XML anlegen,
 - Tasks/Subtasks/Varianten anlegen und ändern,
 - Löschen,
 - Speichern und Laden,
 - Klausur + Musterlösung erzeugen.

Wichtige Stellen im Code: - Datei laden/neu: `HomeController.java:37, :65, :74` - Persistenz/Autosave: `TaskXmlStore.java:73, :348` - Generierung: `HomeGenerateFlow.java:362, :402, :408`

3) Qualität zeigen (5–6 Minuten)

- Tests laufen lassen: `mvn -q test`
- Zwei Beispiele hervorheben:
 - XML-CRUD inkl. Fehlerfälle (`TaskXmlStoreTest.java`)
 - 1000er-Zufallstest (`PostageCalculatorRandomizedTest.java:18`)

4) Javadoc zeigen (2 Minuten)

```
mvn -q -Dmaven.repo.local=/tmp/m2 -Dmaven.compiler.release=21 javadoc:javadoc
```

Dann `target/reports/apidocs/index.html` öffnen.

5) Special Features (5 Minuten)

- Splashscreen + sauberes App-Icon (inkl. Dock): `KlausurCraftApp.java:66, :108, :152, :228`
- Theme-Wechsel in beiden Fenstern: `ThemeService.java:24, :31, :43, :62` plus `HomeGenerateFlow.java:327`
- Suche wird beim Start nicht automatisch fokussiert: `HomeTopBarController.java:26`
- Benachrichtigungen überdecken keine Inhalte: `home.fxml:17` und `HomeNotifications.java:22`
- UI/UX-Regeln im Code:
 - Empathy: `PoolTaskCell.java:90`
 - Guidance: `HomeGenerateFlow.java:221`
 - Novelty: `HomeGenerateFlow.java:284`
- Easter Egg im PDF:
 - Einbau auf Cover: `PdfExportService.java:175`
 - Zufallssprüche: `PdfExportService.java:348`

Moderationshilfe (wenn ich nervös bin)

- “Ich starte kurz mit den Anforderungen und gehe dann direkt in die App.”
- “Danach zeige ich die Tests und die technische Doku (Coverage/Javadoc).”
- “Zum Schluss zeige ich die besonderen Features mit den relevanten Code-Stellen.”

Falls die Zeit knapp wird

Zuerst kürzen: Metriken und Diskussionsbeiträge. Nicht kürzen: Live-CRUD, PDF-Generierung, Theme-Sync, UI/UX-Prinzipien, Easter Egg.